

Benchmarking Quantum Error Correction Decoders on the Surface Code

Thomas Van Camp
April 22, 2026

Quantum error correction (QEC) is our key to unlock the era of fault tolerant quantum computing [1]. By representing logical qubits with groups of physical qubits, errors can be suppressed at an exponential rate as more repetition is added. The error correction process is not easy, however. Errors must be detected, decoded, and corrected, with each step coming with its share of challenges. One of the most important parts of this process is the decoding algorithm used to turn error syndromes into corrective actions. This process must be both accurate and fast as a faulty decoder would defeat the purpose of correcting errors and a slow one would lead to a backlog of measurement information that would ruin any advantage of using a quantum computer [2]. In this work, I examined four different algorithms for decoding on the surface code, a popular option for choice of error correction code. My selections were the Minimum Weight Perfect Matching, Union-Find, Tensor Network, and Neural Network decoders since they are some of the most widely used and studied algorithms. For each algorithm, I attempted to both validate my findings with similar studies and pit each algorithm against one another to determine the advantages and disadvantages of each. The specific metrics I tested were correctness, latency, threshold, robustness, and scalability with Union-Find performing best in the correctness, threshold, and scalability experiments, Neural Network performing best in the latency tests, and Neural Network performing best in the robustness test. The results in this report suggest there are legitimate use cases for each of the decoding algorithms despite their certain limitations either in the near or long term. This is beneficial to the quantum computing community as a whole because no matter which direction we move in, at least one of these decoders will be helpful in pursuing fault tolerance.

1.0 Introduction

Quantum computers have been theorized to provide massive benefits in other scientific fields like chemistry, materials science, cryptography, and AI. However, as of now, quantum bits and gates have not achieved high enough fidelities to realize practical applications on their own. Gates currently have around 99.9% fidelity and qubits have decoherence times on the scale of 100 microseconds to 10 seconds depending on the hardware implementation (superconducting and trapped ion qubits, respectively). These metrics aren't enough to achieve any realistic computation, but, similar to classical computers, repetition can be used to counteract the inconsistencies associated with quantum computers.

Quantum error correction (QEC) is the process of protecting quantum information from noise such as gate errors, decoherence, and measurement errors through the use of error correction codes. Essentially, an error correction code is a form of repetition along with corresponding gates and measurements that enables the system to detect errors that occur during the execution of quantum algorithms. Since quantum information is lost upon measuring a quantum state, special procedures must be taken to perform operations onto ancilla qubits that let us measure certain

properties of the state (“observables”) without disrupting the actual information. From these measurements, it is possible to reason about if or what errors occurred depending on the specific code being used. The process of turning the measurements, or error syndromes, into actual corrective operations is called decoding, and is the main focus of this report. The specific code I will be using is the surface code, a very well researched and widely used code due to its leniency for error rates and high locality, making it easy to implement on hardware. Therefore, I had to choose decoders that were compatible with the surface code to successfully compare them against each other. The four I decided on were Minimum Weight Perfect Matching (MWPM), Union-Find (UF), Tensor Network (TN), and Neural Network (NN). I chose them because they are all very popular choices for decoding the surface code depending on what the associated hardware looks like. Each decoder has had much research done on them to find further optimizations and compatibilities, but due to the breadth of use cases and all of these individual projects, it can be difficult to actually pinpoint the benefits of each relative to each other. Oftentimes, research done on one decoder simply identifies improvements over previous iterations or products using the same decoding algorithm, but fails to address its improvements compared to the field as a whole.

Through this research, I plan to sharpen the line between these seemingly distant algorithm families. I intend to display their performance on an equal playing field in order to expose the pros and cons of each. I then want to validate their metrics with other studies to see if my testing environment is capable of realizing practical QEC decoding, thus either substantiating or detracting from certain aspects of my experiments. Additionally, within each decoder, I plan to determine their unique tradeoffs and how they correspond with performance on different hardware platforms.

2.0 Background

This section will provide a technical overview of different parts of my research to give a foundation for understanding my work as well as motivation for why this research is important in the overall field of quantum computing.

2.1 Quantum Error Correction

The process of correcting errors during a quantum algorithm has many steps that all need to work efficiently in order to provide substantial results. First, controlled operations are performed between the physical qubits and ancilla qubits of the QEC code. This step can introduce errors because, as we already know, gates can produce errors; this is known as the physical error rate. Next, depending on the specific decoder, parity checks or simple measurements are done on ancilla qubits to get an error “syndrome”. These measurements can also introduce errors to the system. This step creates a list of ones and zeros that will be used in the decoding step. Decoding takes place in a classical system, taking in the measurement results and outputting the expected error set. Depending on the algorithm used, you can be more or less likely to receive the actual error set that occurred in your system. If the decoder can’t successfully counteract the error(s) that happened, that is considered a logical error. The final step is to simply apply counteracting operations onto the logical qubit to return it to its original state. This whole process has many

sources of error, so having high fidelity qubits and gates is essential to seeing significant benefits from performing error correction.

2.2 The Surface Code

The surface code is a very popular choice among error correction codes due to its high error threshold and locality. It also has an important characteristic that as the code distance (amount of physical qubits the lattice is across) increases, the logical error rate gets suppressed at an exponential rate.

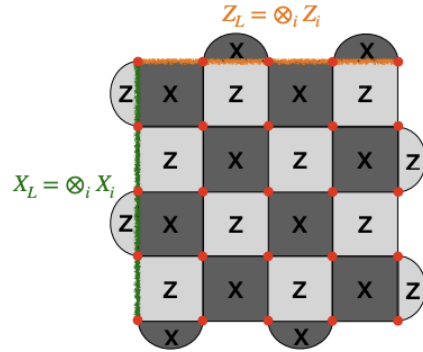


Figure 1. Logical qubit represented with distance 5, rotated surface code. Red dots symbolize physical qubits. [3]

To see this error rate suppression, the physical error rate of the system must be below the code threshold. The threshold is a measure of how high of a physical error rate a decoder can withstand before seeing worse performance from larger code distances. Intuitively, having more repetition decreases the likelihood of getting logical errors, however extra repetition comes with more overhead which can cause more errors. Thus, at some physical error rate, the benefit of more repetition is outweighed by the cost of more error sources. The surface code has a high inherent threshold among QEC codes, which is one reason for its popularity.

The surface code also has high locality. To obtain error syndromes, each physical qubit has two to four controlled gates applied to it (a mix of CNOTs and CZs) to achieve a set of parity measurements for both X and Z type errors. These parity checks are then sent to a decoder to determine the most likely set of errors that occurred. Due to the simplicity of this detection scheme, it is among the easiest QEC codes to implement on hardware, further justifying the surface code's popularity.

2.3 Decoding Algorithms

In this section, I will provide a brief overview of each decoding algorithm I intend to experiment with so one might understand why their observations line up with the results of the tests.

2.3.1 MWPM

Minimum weight perfect matching decoders are seen as the standard, baseline decoders. They are very commonly seen with topological codes that can be represented as graphs (like the surface code).

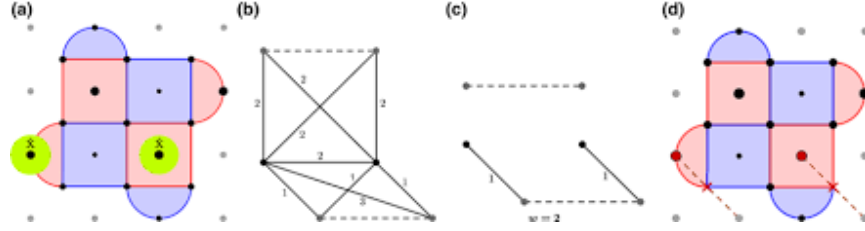


Figure 2. Error correction using a MWPM algorithm. Error detections are highlighted in green, and a minimum weight subgraph is constructed to determine the most likely set of errors, given by the red axes.

These algorithms work by representing the error correction code as a graph where each ancilla qubit is a node. The edge weights are based on the likelihood of errors occurring in conjunction (affected by distance between them and qubit/system reliability). The algorithm then finds the minimum weight subgraph to connect all nodes that detected an error, and can use the edges of the graph to execute the corresponding corrective measures.

2.3.2 Union-Find

Union-Find decoders can be seen as an approximate extension of MWPM decoders. They are also commonly seen with topological codes, but have recently been adapted to other low density parity check codes [4].

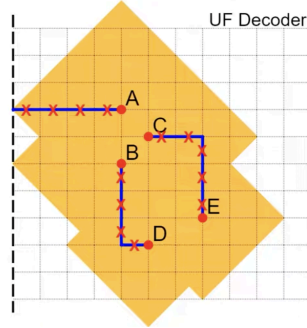


Figure 3. Finished state of UF decoder. Yellow region represents the error cluster and blue lines with red axes represent the chain of errors connecting flagged ancilla qubits [5].

These algorithms work by using the union-find data structure to systematically merge error “clusters”. For each ancilla qubit that detects an error, the algorithm begins growing a cluster around it. At every step, clusters have the chance of running into each other or the code boundary. If another cluster is touched, they merge and will only continue growing if there are an odd number of errors in the cluster. If the cluster reaches the boundary or touches a cluster that results in the merged cluster touching the boundary or having an even number of errors, the cluster will not grow. Once all clusters have stopped growing, all the encapsulated errors within each cluster are connected with a chain of errors (connecting to the boundary if also touched). These then become the corrective operations used to fix the logical qubit.

2.3.3 Tensor Network

Tensor network decoders are another family of decoding algorithms that specialize with 2D and 3D topological codes. They are generally seen as the most accurate type of decoder, often offering near optimal performance.

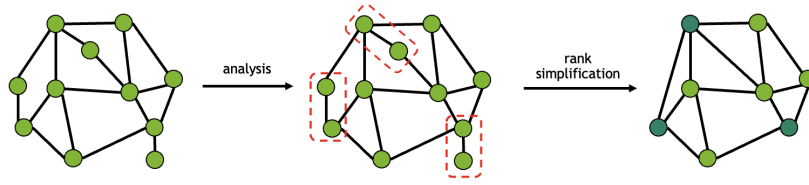


Figure 4. The tensor network decoder analyzes correlations between error correction terms and prunes unlikely or impossible groupings to contract the network [6].

These decoders work by first mapping all of the error correction elements to nodes in a tensor network. These elements include the data qubits, measurement results, gates, noise model, and more. Then, it examines nearby nodes (representing correlated events), and determines their likelihood relative to the overall network. The decoder contracts the network by pruning unlikely events, leaving the most likely outcome as the remaining network. This can then be parsed to achieve the corrective operations.

2.3.4 Neural Network

The last decoding algorithm I plan to test is a neural network. While not strictly an algorithm, neural networks provide very efficient implementations for solving complex problems, making them useful for many different error correction codes, not just topological ones. These decoders work by training a model on syndrome-error pairs. After training it on enough samples, the model can quickly receive new syndrome information and respond with a set of errors that it thinks is most likely to occur. This is the only decoder of the four that requires pre-computation, so all of its results should be understood with that caveat in mind.

2.4 Motivation

Much quantum computing research has been put into creating better QEC decoders [7-10]. This is even more so the case for decoders on the surface code. All of these families of decoding algorithms have many iterations of tools that have their own unique quirks that demonstrate some interesting tradeoffs between each other. However, between these algorithmic families, there is very little cost-benefit analysis being done. Most researchers are focused on improving the one specific decoder they are working on without seeing how other decoders compare, or may even be more useful in their situation. Additionally, these decoders are all being tested and worked on in different environments and on different hardware, which can make comparisons systematically faulty.

Through my research and findings, I intend to demonstrate a clear comparison of these different types of decoding algorithms by running various tests on them in the same environment. I want my results to indisputably show how the decoders stack up in certain aspects and compare to one another. Every decoder has strengths and weaknesses that need to be addressed either through further research or correct usage depending on system hardware, and I want to determine situations where each can be effectively utilized. In doing so, I hope my research can inform others about the best decoders to use no matter where quantum computing goes in the future, so we can hopefully reach fault tolerance and start realizing practical quantum computation.

3.0 Methods

This section of the report will focus on the experimental setup for testing the decoders in the same environment. It will also go over the specific libraries used, tests implemented, and procedure for running each of the tests.

3.1 Simulation Framework

To effectively test QEC decoders, you need two things: 1) a quantum circuit you would like to experiment with, and 2) a way to efficiently sample running that circuit millions of times, hopefully within a few seconds [11]. This is not an easy obstacle to overcome, but thankfully I'm not alone. Google has developed a very fast simulation library named Stim [12], that has a wide range of functionalities. For my research, I will be focused on the built-in "surface_code:rotated_memory_x" circuit along with the different noise models they allow including bit/phase flip, depolarization, measurement, and reset errors. Besides being able to change the inherent noise of the circuit, I can also specify the code distance, which will be necessary for many of the tests. Using Stim not only allows me to generate the surface code with my desired parameters, but also sample running it millions of times in seconds. This is important because it will enable me to test a lot of error samples in a reasonable amount of time, with the main bottleneck being decoding instead of circuit generation and simulation. The simulation interface provides results in the form of error syndromes and observed logical errors, which makes is very easy to decode samples and determine if the decoders are correctly identifying logical errors for each individual simulation.

3.2 Specific Decoders

For the minimum weight perfect matching decoder, I decided to go with PyMatching [13-14]. This is a well-maintained, easy to use Python library that integrates seamlessly with Stim. This made it perfect for testing in my environment, especially as sort of a baseline decoder. It implements a version of the Sparse Blossom algorithm to quickly decode syndrome information.

For the union-find decoder, I ended up deciding to use a Python library called ldpc [15]. This decoder was the most difficult to implement as many other libraries that had union-find decoders didn't have any compatibility with Stim, often utilizing their own simulation frameworks. While not directly compatible with Stim, I was able to create a wrapper class that could translate all the information necessary between Stim and ldpc's UnionFindDecoder class. The library uses an implementation of the Delfosse-Nickerson union-find decoder.

For the tensor network decoder, I used NVIDIA's cudaq Python library [16]. I again had trouble finding a library to implement a decoder in my native environment, so while this decoder is highly regarded and well-maintained, it did require certain hardware requirements that I lacked. However, the documentation comes with a Docker container that users can download to use the library with insufficient hardware. This is how I was able to experiment with their tensor network decoder, so the results may be slightly different than if I ran it on my computer locally. Their implementation creates the tensor network from the Tanner graph of the surface code. Finally, for the neural network decoder, I used Python's PyTorch library to create and run all my models. As one of the most well known tools for machine learning, I figured it would work great

for my purposes. Since the models are fed on whatever input I feed it, it can be used with whatever simulation tool I use (in this case Stim).

3.3 Evaluation Metrics and Procedures

In this section, I will describe the five tests I performed on each decoder, what they are measuring, and how they were implemented.

3.3.1 Correctness

One of the most important qualities of a QEC decoder is its correctness. This is simply a measure of how often the decoder correctly predicts a logical error has occurred given an error syndrome. However, at the distances and physical error rates I will be testing, it is frequent that no errors occur in the circuit. Therefore, to get a better understanding of these decoders' abilities, I will be studying their conditional correctness. This changes the samples that I examine from all to only ones where logical errors actually occurred. From there, I can see how well the decoders perform when actual errors are happening. To implement this, I generate surface codes with a range of physical error rates (usually 10^{-4} - 10^{-2}) with distance 5, and sample each around 100,000 times. From those samples, I can see which ones produced logical errors, and compare my decoder's prediction with the observed result. I divide the number of incorrect predictions by the number of times a logical error occurred to achieve the conditional error rate.

3.3.2 Latency

Another important metric for a decoder is its latency. This might be the most important factor if a decoder is to be used to achieve fault tolerance, as real time decoding is necessary to correct errors during algorithm execution. To implement this test, I generate surface codes with a range of physical error rates (usually 10^{-4} - 10^{-2}) with distance 5, and sample each around 10,000 times (or less depending on how slow the decoder is). From there, I start a timer, decode the whole batch of samples, and stop the timer. I can then calculate the average decoding latency by dividing the total time by the amount of samples I generated.

3.3.3 Threshold

The next metric I want to test for each of my decoders is their threshold. Again, the threshold is the physical error rate at which increasing code distance does not yield exponentially lower logical error rates. To implement this test, I iterate over a range of code distances (3, 5, 7, and 9), and for each, I iterate over a range of physical error rates depending on the decoder. For each of these distance-error rate pairs, I create a surface code and sample it around 100,000 times. From these samples, I can find how many times my decoder's prediction disagreed with a logical error actually occurring or not. The proportion of incorrect predictions to the total amount of samples is the logical error rate. Thus, below the decoder's threshold, you would expect higher code distances to achieve lower logical error rates and the opposite to occur above the threshold. This ideally results in a plot where the lines tracking physical versus logical error rate for each code distance cross over neatly around some specific physical error rate (the threshold)

3.3.4 Robustness

Another characteristic of decoders is how robust they are against different types of noise. To test how resilient they are against different noise models, I devised 5 different sub-tests for each

decoder. Each would start by creating a surface code with different types of noise; 1) Control = only bit/phase flip errors, 2) Control + Depolarization errors, 3) Control + Measurement errors, 4) Control + Qubit Reset Errors, and 5) Control + Depolarization, Measurement, and Reset errors. From there, I ran the same conditional correctness test as before on each surface code for a range of physical error rates. This would give a good measure for how suitable each of the decoders are for noisy, intermediate scale quantum computing (NISQ).

3.3.5 Scalability

Finally, I want to test each decoder on scalability. Specifically, this is testing to see how their decoding latency scales as code distance increases. To implement this, I will simply run one of the aforementioned latency tests for a range of code distances (3, 5, 7, and 9) on every decoder. This will determine how efficient these decoders will be once we can scale the hardware enough to start seeing low enough logical error rates to realize practical quantum computing applications.

4.0 Results

Equipped with all the tools and background necessary to perform the desired experiences, I will now present the results of my tests for each decoder and elaborate on any important deviations from the outlined process.

4.1 Correctness and Robustness

All of the decoders showed similar trends for conditional correctness; as physical error rates increased, conditional error rates also increased.

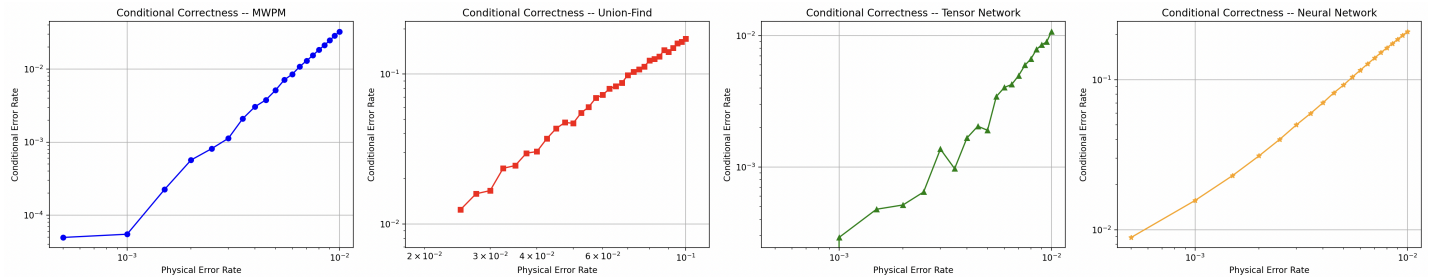


Figure 5. Physical error rate vs. Conditional error rate plots for all four decoders (MWPM, UF, TN, and NN, respectively).

By extrapolating slightly, it appears the union-find and tensor network decoders have the lowest conditional error rates with the neural network definitively performing the worst. However, in the robustness test, the performances of each decoder get shaken up.

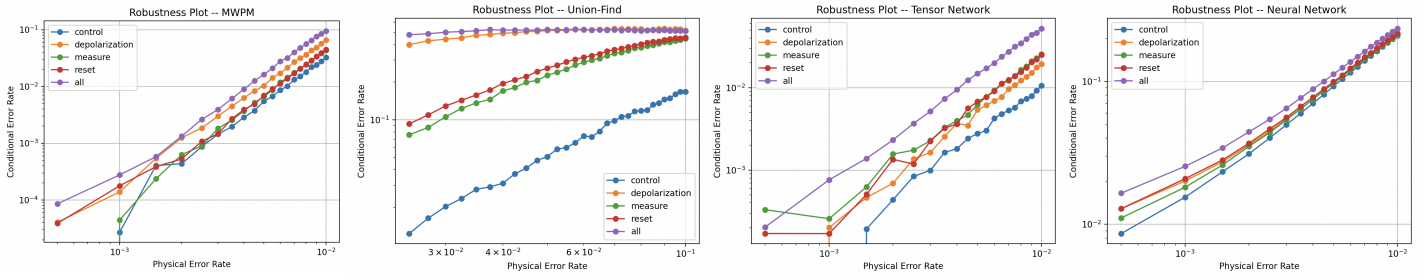


Figure 6. Physical error rate vs. Conditional error rate plots for all four decoders (MWPM, UF, TN, and NN, respectively). Each plot has one line for each error model; blue = control (only bit and phase flip errors), orange = control + depolarization errors, green = control + measurement errors, red = control + measurement errors, and purple is all four error sources combined.

Despite performing best in correctness, the union-find decoder can not seem to handle any amount of other error. In fact, all the lines start to approach the value 0.5, suggesting the decoder can only do as good as randomly guessing if a logical error occurred or not. And despite performing the worst in correctness, the neural network plot suggests the model is rather undeterred when additional noise sources are introduced.

4.2 Latency and Scalability

Decoding latency may be the test that saw the most discrepancy between the decoders, with each decoder having very noticeable benefits and costs.

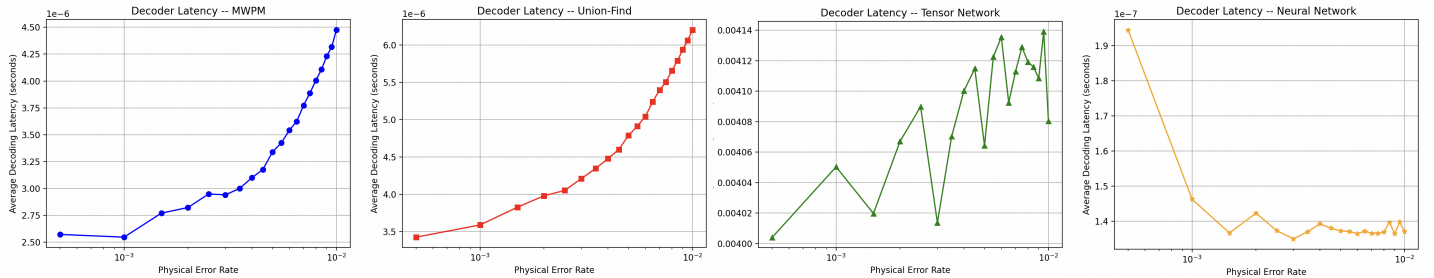


Figure 7. Physical error rate vs. Average decoding latency plots for all four decoders (MWPM, UF, TN, and NN, respectively).

The obvious disparities in these plots are the scaling and base values. The neural network decoder is the fastest by an order of magnitude, and the tensor network is the slowest by many. The MWPM and union-find decoders are nearly identical; both scaling exponentially taking around $3\mu s$ with lower physical error rates. On the other hand, the tensor and neural network decoders both have a (essentially) constant decoding latency, no matter the error rate. Similar trends continue to appear in the scalability results.

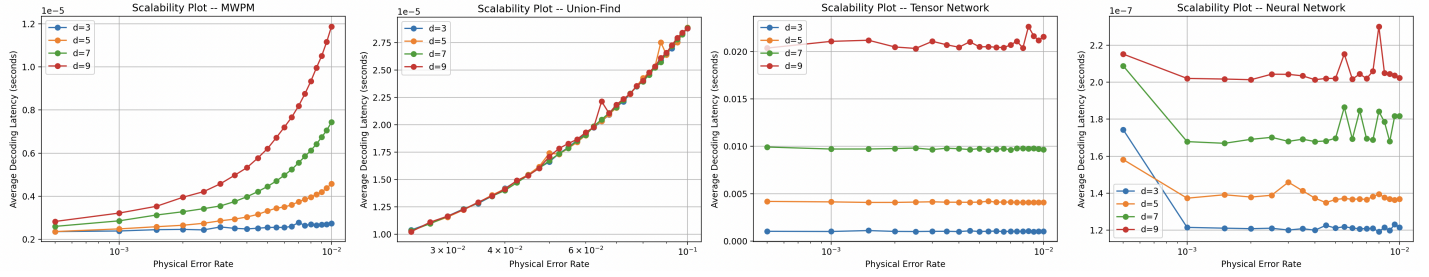


Figure 8. Physical error rate vs. Average decoding latency plots for all four decoders (MWPM, UF, TN, and NN, respectively). Each plot has one line for each tested code distance; blue = 3, orange = 5, green = 7, and red = 9.

MWPM decoding latency appears to scale exponentially not only with physical error rates, but also with code distance, whereas the union-find decoder doesn't appear to have any changes in latency at the higher distances. The tensor and neural network decoders continue to demonstrate constant time decoding, but both scale exponentially with code distance (with tensor network appearing to scale a bit faster).

4.3 Threshold

The final test performed was the threshold test. Ideally, a decoder has a higher threshold so that it can withstand a higher error rate before seeing worsening accuracy as code distance increases.

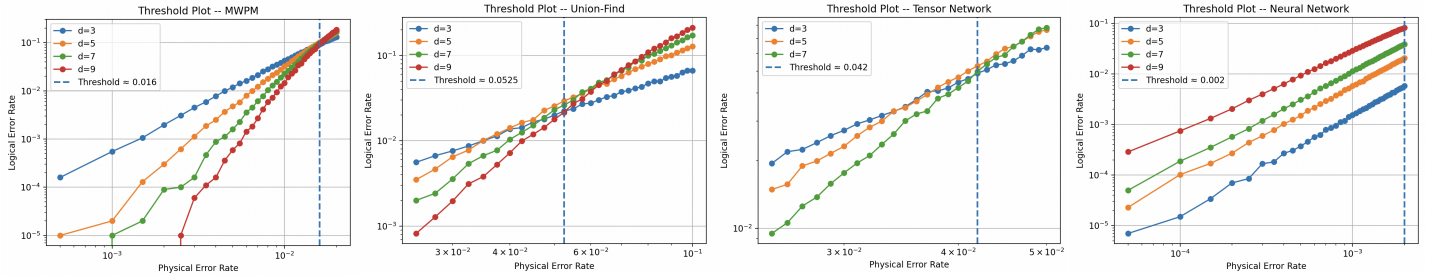


Figure 9. Physical error rate vs. Logical error rate plots for all four decoders (MWPM, UF, TN, and NN, respectively). Each plot has one line for each tested code distance; blue = 3, orange = 5, green = 7, and red = 9. The blue dashed line represents the estimated threshold value based on the results.

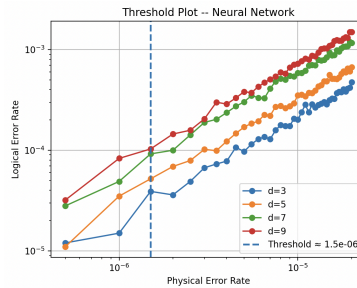


Figure 10. Physical error rate vs. Logical error rate plot for the neural network decoder at a lower physical error rate range. Notably no distinct crossover is seen.

The results for the MWPM, UF, and TN decoders are all reasonable, with UF having the highest threshold of around 5.2% and TN right behind it with 4.2%. The neural network however did not appear to have a threshold at any physical error rate I tested, suggesting a systematic error.

5.0 Discussion

This section will discuss the findings found in the results section for each decoder and address any limitations I incurred while trying to come to run these tests. It will also suggest reasons each decoder might be valuable to other quantum computing research.

5.1 MWPM

The MWPM decoder performed similarly to expected results [7]. It performed reasonably well in all of the tests, standing out in latency and robustness. As a rather simple decoder that is often the benchmark for others, that is what I would expect. It performs well enough to identify most errors at physical error rates possible with current hardware, and can do so at very fast speeds. It has a slightly lower threshold than some of the other decoders, so it might not be the best choice for all quantum systems. The best use cases for a MWPM decoder will be in the future when hardware becomes even more reliable so that physical error rates are even lower and we hopefully don't need as large of code distances as we currently anticipate needing for realistic quantum applications.

5.2 Union-Find

The UF decoder also performed similarly to expected results [8]. It performed very well in every test except the robustness test, standing out in threshold, scalability, and correctness. It performed on par with the tensor network decoder in terms of accuracy, which is considered a very optimal decoder. Additionally, it achieved the sublinear latency relative to code distance suggested by other sources [8]. The only shortcoming was that its observed threshold of 5.2% was slightly below ideal results (around 9%). This is likely due to not having the specific parallelism and scheduling tools necessary to optimize decoding performance. There are many use cases for union-find decoders now that they are being adapted to more QEC codes, so the only real limiting factor I can see is any system that suffers from many distinct noise sources. Therefore, this decoder will likely become more and more popular as hardware improves, even if very large code distances are required.

5.3 Tensor Network

The tensor network decoder also saw results similar to expected [9]. It performed best in the accuracy and threshold tests, achieving near the best in both. The obvious downside to this type of decoder is its much higher decoding latency, taking on the scale of milliseconds instead of microseconds. However, this can be outweighed by the fact that it decodes in constant time relative to physical error rates. One unique limitation seen during testing with this decoder was that I couldn't simulate the distance 9 surface code for the threshold test due to hardware constraints. I needed to run this decoder in NVIDIA's provided Docker container to access the cudaq Python libraries, and it lacked the memory requirements to run this specific test. Additionally, the reason for needing the container was that my laptop lacks GPU capabilities that the library requires. I specified in my code to not use the GPU accelerated performance, but I'm unsure if there were other unseen optimizations being made that couldn't be done on my laptop natively. The use cases for a tensor network are much more limited than some of the other decoders. In my eyes, the best case scenario is in a system that uses trapped ions (so the decoding bottleneck is replaced by operation latencies), that suffer from lots of noise and high physical

error rates, even at high code distances. This would allow the decoder to boast its high threshold and good robustness while not being a bottleneck in the overall system's performance.

5.4 Neural Network

Finally, the neural network decoder was an interesting case. In some aspects, it performed remarkably well such as in latency, scalability, and robustness. However, in my case, it is difficult for those advantages to account for the poor performance in the accuracy and threshold tests. Having the worst accuracy and no observable threshold suggests that this decoder will perform the worst of the bunch with current hardware and as code distances start increasing. These errors do appear to be systematic, though, and I believe that the core issue was overfitting my models. At the distances and error rates I was experimenting with, there are only so many error syndromes that are bound to show up. Therefore, training my model with any amount of data (I used 1000 simulations), is likely to cause all likely syndromes to be seen and accounted for in the model. Therefore, there is no room for interpretation and the model will simply choose the outcomes that appeared most often for any syndrome. I think with better training practices from someone with more neural network experience than myself, this type of decoder could become very useful. In an ideal scenario, it could be useful in any QEC scheme as it is compatible with any type of input and can output responses in almost no time. Additionally, its resistance against different error types means it could be used in near term applications on NISQ computers and excellent scalability suggests it could be used in the long term as code distances get arbitrarily large.

6.0 Conclusion

Through this work, I have shown the unique advantages and disadvantages of multiple QEC decoders on the surface code. These traits of the decoders allow for them to be useful in their own distinct circumstances, ensuring that researchers will always have a viable decoder to use no matter where quantum hardware ends up in the future. To improve this study, I would like to work with somebody more experienced in neural networks so that we can effectively benchmark that decoder in particular. Additionally, I would perform all of these tests on a more capable machine that has the computing power to simulate higher code distances for more iterations and faster. Finally, I would also like to run these same tests on other QEC codes to get an even better understanding of how different codes and decoders can improve quantum computing. I hope this research demonstrates the importance of getting a wider perspective on the field of QEC decoders and encourages others to do the same either through their own research or simply reconsidering their alternatives. I believe that decoders will play a non-negligible role in our push towards fault tolerance and that similar research may be essential to getting us there.

Source code for this study can be found at https://github.com/tvc04/qec_decoding

References

- [1] Google Quantum AI et al., "Quantum error correction below the surface code threshold," 2024, arXiv:2408.13687.
- [2] F. Battistel et al., "Real-Time Decoding for Fault-Tolerant Quantum Computing: Progress, Challenges and Outlook," 2023, arXiv:2303.00054.
- [3] Z. Cheng, E. Huang, V. Khemani, M.J. Gullans, and M. Ippoliti, "Emergent unitary designs for encoded qubits from coherent errors and syndrome measurements," 2024, arXiv:2412.04414.
- [4] N. Delfosse, V. Londe, and M. Beverland, "Toward a Union-Find decoder for quantum LDPC codes," 2021, arXiv:2103.08049.
- [5] Y. Wu, "APS2022 Interpretation of Union Find Decoder on Weighted Graphs and Application to XZZX Surface Code," 2022.
<https://www.youtube.com/watch?v=BbhqUHKPdQk>.
- [6] NVIDIA, "Overview — NVIDIA cuQuantum," 2023.
<https://docs.nvidia.com/cuda/cuquantum/latest/cutensornet/overview.html>.
- [7] O. Higgott, "PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching," 2021, arXiv:2105.13082.
- [8] N. Liyanage, Y. Wu, S. Tagare, and L. Zhong, "FPGA-based Distributed Union-Find Decoder for Surface Codes," 2024, arXiv:2406.08491.
- [9] A. Kaufmann, and I. Arad, "A blockBP decoder for the surface code," 2024, arXiv:2402.04834.
- [10] Y. Zhu, C. -Y. Peng, Y. H. Chen, S. Niu, Y. -L. Ueng and D. Wu, "Syndrilla: Simulating Decoders for Quantum Error Correction Using PyTorch," 2025 IEEE International Conference on Quantum Computing and Engineering (QCE), doi: 10.1109/QCE65121.2025.10470.
- [11] C. Gidney, "Stim: a fast stabilizer circuit simulator," 2021, arXiv:2103.02202.
- [12] Google LLC, "Stim," 2025. [Online]. Available: <https://github.com/quantumlib/Stim>.
- [13] O.Higgott, C. Gidney et al., "PyMatching," 2025. [Online]. Available: <https://github.com/oscarhiggott/PyMatching>.
- [14] O. Higgott, and C. Gidney, "Sparse Blossom: correcting a million errors per core second with minimum-weight matching," 2023, arXiv:2303.15933.
- [15] R. Joschka et al., "LDPC: Python tools for low density parity check codes," 2022, <https://pypi.org/project/ldpc>.
- [16] NVIDIA, "cuda-quantum," 2026. [Online]. Available: <https://github.com/NVIDIA/cuda-quantum>.